

Copione Presentazione: Metodi Numerici per Scientific Computing

Slide 1: Titolo

"Buongiorno a tutti. Presento oggi il mio progetto finale per il corso di Numerical Methods for Scientific Computing, intitolato 'Modellazione e Scoperta Parametrica della Propagazione Epidemica'. L'obiettivo del mio lavoro è stato quello di mettere a confronto le prestazioni, la scalabilità e l'accuratezza dei metodi numerici classici contro un approccio puramente data-driven basato sull'Intelligenza Artificiale, nello specifico le Physics-Informed Neural Networks (PINN)."

Slide 2: L'Obiettivo del Progetto

"Il progetto nasce nel contesto della prima ondata di COVID-19 in Italia (Feb-Mag 2020), un periodo ideale per isolare una dinamica di diffusione spaziale 'pura'.

L'obiettivo ingegneristico del mio lavoro è stato quello di sviluppare un'infrastruttura data-driven completa, pronta ad acquisire ed elaborare i dati reali della Protezione Civile tramite un'equazione differenziale spaziale (PDE).

Tuttavia, trattandosi di un esame di Analisi Numerica, oggi il nostro focus non sarà l'epidemiologia, ma la validazione algoritmica. Vedremo infatti come l'infrastruttura geografica sviluppata venga utilizzata per testare i solutori classici e le reti neurali in un ambiente matematico rigorosamente controllato (tramite una soluzione fabbricata). Questo ci permetterà di misurare le prestazioni, la stabilità e gli errori degli algoritmi con precisione assoluta."

Slide 3: L'Acquisizione dei Dati (Protezione Civile)

"Per la fase di data engineering, ho sviluppato uno script che scarica dinamicamente il dataset direttamente dalla repository GitHub ufficiale della Protezione Civile. Questo approccio automatizzato evita di archiviare file CSV statici e previene discrepanze nei dati.

Abbiamo applicato un filtraggio temporale rigoroso, isolando esattamente le 10.379 registrazioni della Prima Ondata (Feb-Mag 2020). Durante questo processo, è stato cruciale associare i contagi alle corrette coordinate geospaziali (Latitudine e Longitudine). Ho dovuto filtrare via i dati con coordinate impostate a zero (i casi 'in fase di definizione' della Protezione Civile) per evitare che il modello collocasse migliaia di focolai in mezzo all'Oceano Atlantico. Inoltre, ho dovuto

implementare una correzione matematica del Timezone UTC nativo nei dati governativi. I filtri temporali, infatti, si basano su date 'naive' (senza fuso orario), e se non avessimo rimosso l'etichetta UTC originale il codice Pandas sarebbe andato in crash bloccando l'elaborazione. Un passaggio chiave è stata la mappatura di queste coordinate su una griglia normalizzata $[0,1] \times [0,1]$. Usare coordinate adimensionali in scala 0-1 è vitale per due motivi: nel Deep Learning impedisce che i gradienti esplodano, e nei metodi numerici semplifica enormemente il calcolo dei passi spaziali 'h'. Normalizzando lo spazio, infatti, 'h' diventa una semplice frazione costante (ad esempio 1 diviso 50) identica per gli assi X e Y, evitandoci di dover calcolare complessi differenziali basati sui reali chilometri terrestri.

Per la discretizzazione ho scelto una griglia 50x50. Non è un numero casuale: una griglia 10x10 avrebbe distrutto la risoluzione, mentre una griglia 1000x1000 avrebbe generato milioni di celle inutili, saturando la RAM. Con la griglia 50x50, le 107 province italiane ufficiali diventano 105 coordinate attive. Questo accade perché due precise coppie di province, ovvero Firenze con Prato e Messina con Reggio Calabria, essendo geograficamente vicinissime, collassano nello stesso pixel matematico, fungendo logicamente da un unico grande focolaio virale a causa dell'intenso pendolarismo."

Slide 4: Il Modello Matematico

"Entriamo nel cuore del Percorso Numerico Classico. L'equazione di Diffusione Spaziale che modella il nostro sistema è la derivata prima di u rispetto al tempo uguale a D per il Laplaciano di u ($\partial u / \partial t = D \nabla^2 u$). In questa equazione, ' u ' rappresenta i contagiati.

Ma cos'è esattamente il Laplaciano? Matematicamente, è un operatore differenziale dato dalla somma delle derivate parziali seconde spaziali. Fisicamente, misura la 'concavità' o la differenza di concentrazione in un punto rispetto all'ambiente circostante: se una provincia ha un picco di contagi rispetto a quelle vicine, il Laplaciano agirà come una forza termodinamica che spinge il virus a disperdersi verso l'esterno per riequilibrare il sistema.

Il parametro D , che moltiplica questo operatore, è l'incognita macroscopica del sistema. Questo è fondamentale: non stiamo calcolando quanti contagi ci saranno domani (forward problem), ma stiamo cercando di scoprire quale parametro fisico ha generato i dati che abbiamo già osservato. Si tratta quindi di un 'Inverse Problem' matematico."

Slide 5: Percorso Numerico (Discretizzazione)

"I computer non sanno risolvere equazioni differenziali continue, quindi la PDE deve essere tradotta in un grande Sistema Lineare Sparso ($Au = b$). Per discretizzare lo spazio ho applicato il Metodo alle Differenze Finite.

Per farlo, la matematica ci impone di usare lo Sviluppo in Serie di Taylor: esprimiamo il valore dei contagi in un punto guardando il punto successivo e quello precedente. Troncando questo sviluppo infinito esattamente al secondo ordine, scartiamo i termini di errore infinitesimi e trasformiamo la derivata continua in una semplice sottrazione algebrica calcolabile dal processore. Applicando questa tecnica sia sull'asse X che sull'asse Y, nasce il famoso 'Stencil a 5 Punti'. Di cosa si tratta? È una struttura geometrica a croce che detta le regole di scambio spaziale. Il nodo centrale, ovvero la provincia infetta che stiamo calcolando, assume nella matrice un coefficiente di -4, in quanto rappresenta la sorgente da cui il virus defluisce. I 4 nodi cardinali immediatamente adiacenti (Nord, Sud, Est, Ovest) assumono un coefficiente di +1, poiché assorbono il contagio in parti uguali. Ai bordi della mappa ho imposto condizioni di Dirichlet nulle per impedire al virus di 'teletrasportarsi' da un capo all'altro della memoria.

Applicando questo stencil a tutti i pixel della griglia 50x50, si genera una matrice A di 2500x2500, per un totale di oltre 6.2 milioni di celle. Qui entra in gioco la 'Curse of Dimensionality' in RAM: di queste 6 milioni di celle, solo 12.300 contengono numeri, le altre sono zeri. La sparsità è del 99.8%. L'archiviazione in matrici CSR (Compressed Sparse Row) salva il sistema. Invece di allocare la matrice vuota, il CSR usa tre piccoli vettori monodimensionali per annotare solo i valori non nulli e le loro coordinate. Questo abbatte drasticamente i consumi di memoria."

Slide 6: Forward Problem (Metodi Iterativi)

"Per la risoluzione iterativa del sistema, l'uso dell'inversa matematica (A^{-1}) è da escludere: causerebbe il fenomeno del Fill-in, riempiendo gli zeri di numeri e mandando in crash la RAM. Ci affidiamo quindi ai Sottospazi di Krylov. Matematicamente, partendo da un errore iniziale e da un'ipotesi iniziale x_0 , calcoliamo il residuo iniziale r_0 , che è definito esattamente come $b - Ax_0$. Lo spazio di Krylov genera vettori moltiplicando ripetutamente la matrice A per questo residuo: r_0, Ar_0, A^2r_0 , e così via. Il vantaggio immenso è che richiedono solo moltiplicazioni Matrice-Vettore, dove il formato CSR salta 6 milioni di zeri ed eccelle a velocità fulminea. Per valutare la qualità della soluzione durante queste iterazioni, non usiamo l'errore assoluto, ma il Residuo Relativo: dividiamo l'errore per la grandezza del bersaglio, perché sbagliare di 100 unità su un milione di contagi è matematicamente trascurabile, ma su 100 totali sarebbe un disastro.

Ho implementato il Gradiente Coniugato (CG) perché è l'algoritmo ottimale per matrici simmetriche definite positive (SPD).

Ma perché moltiplicando il Laplaciano per -1 otteniamo una matrice SPD?

Il nostro operatore Laplaciano discreto ha dei -4 sulla diagonale principale, il che lo rende formalmente una matrice 'definita negativa'. In algebra lineare, perché un metodo come il Gradiente Coniugato possa convergere in sicurezza, la matrice deve essere Definita Positiva, ossia

soddisfare la condizione $x^T A x > 0$ per ogni vettore non nullo. Moltiplicando l'intera matrice per -1, i -4 sulla diagonale diventano +4, invertendo interamente lo spettro degli autovalori da negativi a positivi e trasformandola in una perfetta matrice Simmetrica e Definita Positiva (SPD). Inoltre, 'Simmetrica' significa che lo scambio virale da Prato a Firenze è speculare e identico a quello da Firenze a Prato.

Ho però implementato anche il GMRES. Perché? Per garantire resilienza contro eventuali termini convettivi asimmetrici. Se introducessimo 'il vento' (es. un pendolarismo a senso unico verso est), la matrice perderebbe la simmetria e il CG andrebbe in crash.

Il GMRES, invece, gestisce qualsiasi matrice asimmetrica sfruttando l'iterazione di Arnoldi per costruire una base ortonormale dello spazio di Krylov. Ma cosa significa? Quando generiamo i vettori di Krylov, essi tendono a diventare quasi paralleli tra loro, causando errori di arrotondamento catastrofici per il computer. L'iterazione di Arnoldi (basata sul processo di Gram-Schmidt) interviene proprio per ortogonalizzare i vettori, imponendo che siano perfettamente perpendicolari tra loro (ortogonali) e di lunghezza pari a uno (normali). Costruire questa base ortonormale garantisce la massima stabilità numerica al solutore."

Slide 7: Inverse Problem (Data Fitting)

"Ottenuto il motore risolutivo, dobbiamo scovare il parametro D. Questo richiede la minimizzazione della distanza tra la previsione matematica e i dati empirici reali. È un problema di Data Fitting.

Ho applicato l'Ottimizzazione ai Minimi Quadrati Non Lineari tramite l'algoritmo di Levenberg-Marquardt. Si tratta di un algoritmo ibrido fenomenale che fa da ponte tra due metodi classici, regolato dall'equazione normale smorzata:

$$(J^T J + \lambda I) \delta = J^T r$$

Analizziamo nel dettaglio a cosa fa riferimento ogni singolo termine di questa formula fondamentale:

- **J (Matrice Jacobiana)**: È la matrice delle derivate prime dei residui rispetto al parametro D. Indica la pendenza dell'errore.

- **$J^T J$ (Approssimazione dell'Hessiana)**: È il prodotto della Jacobiana trasposta per se stessa, che approssima brillantemente la matrice delle derivate seconde (l'Hessiana, che descrive la curvatura) evitando il calcolo oneroso e proibitivo in termini di RAM.

- **λ (Parametro di smorzamento o damping parameter)**: È il cuore flessibile dell'algoritmo. Se è molto grande, domina il termine λI e l'equazione diventa simile alla Discesa del Gradiente (passi

piccoli, sicuri e stabili quando siamo lontani dal minimo). Quando invece λ tende a zero, il termine si annulla, l'equazione si riduce a $J^T J \delta = J^T r$ (il metodo di Gauss-Newton) e l'algoritmo garantisce una convergenza quadratica fulminea quando siamo vicini alla soluzione.

- **I (Matrice Identità):** Serve a scalare correttamente il termine di smorzamento lungo la diagonale.

- **δ (Vettore di incremento/passo):** Rappresenta la correzione esatta da sommare al parametro D ad ogni iterazione per avvicinarci al minimo.

- **$J^T r$ (Gradiente della funzione di costo):** È il termine noto del sistema, ottenuto moltiplicando la Jacobiana trasposta per il vettore dei residui r (la differenza tra i contagi simulati e quelli reali).

Un dettaglio fondamentale del codice di test sviluppato è che il valore iniziale e il target di D sono stati scelti in modo del tutto arbitrario (ad esempio fissando la finta realtà a 0.35). Questo non svaluta il test: lo scopo di quello script non era calcolare il Covid, ma isolare e dimostrare che la logica di calibrazione e data fitting converge matematicamente a prescindere dal valore di partenza. Nello step completo, questo valore fittizio viene sostituito dai dati reali della Protezione Civile.

Ingegnieristicamente, il solutore GMRES viene inglobato nell'ottimizzatore. Per evitare 'over-solving' su dati medici reali che sono intrinsecamente rumorosi e per rispettare i limiti del float64, ho impostato una tolleranza mirata di $1e-6$. In sole 9 iterazioni, per un tempo totale di 0.05 secondi, l'algoritmo ha calibrato il vero coefficiente, ottenendo la convergenza a $D=0.3500$."

Slide 8: Il Percorso Deep Learning (PINN)

"Abbandoniamo la matematica discreta e passiamo al Percorso B: le Physics-Informed Neural Networks. Invece di costruire matrici, addestriamo una Rete Neurale. L'Equazione di diffusione viene codificata direttamente nella Loss Function.

L'ottimizzazione diventa un bilanciamento tra due componenti fondamentali:

- **Data Loss:** È la componente di apprendimento supervisionato. Misura l'errore (ad esempio il Mean Squared Error) tra le stime della rete e i dati reali osservati sul territorio (i contagi storici registrati dai sensori). Spinge la rete a imitare fedelmente la realtà empirica.

- **Physics Loss:** È la componente di regolarizzazione fisica. Calcola il residuo dell'equazione differenziale (la PDE di diffusione) inserendo le derivate della rete. Agisce come una severa penalità che punisce la rete ogni volta che questa genera previsioni termodinamicamente o fisicamente impossibili.

Il calcolo delle derivate spaziali è puramente analitico tramite l'Automatic Differentiation (Autograd). Per garantire che le derivate seconde spaziali non si azzerino (come accadrebbe con la

classica funzione ReLU), l'intera rete è stata costruita usando la funzione di attivazione Tangente Iperbolica (Tanh)."

Slide 9: L'Architettura Mesh-Free

"L'impatto più grande è che le PINN offrono un'Architettura Mesh-Free. Come vedete nel grafico, nessuna griglia strutturata è richiesta, annullando il problema della RAM.

La rete apprende l'epidemia interpolando da coordinate stocastiche continue chiamate Collocation Points. Abbiamo 500 punti blu, che sono i dati reali, e ben 2000 punti rossi sparsi casualmente. Su questi punti rossi non conosciamo i contagi, ma forziamo la rete a far quadrare la PDE. Questo approccio garantisce la massima resilienza contro i dati geografici frammentati, impedendo alla rete neurale di 'inventare' focolai impossibili in province dove i sensori reali erano assenti o guasti."

Slide 10: Gradient Pathology e Stallo

"Il problema ingegneristico più grave di questo approccio è lo scontro tra i gradienti dei Dati e della Fisica, fenomeno noto come Gradient Pathology.

Come si vede chiaramente dal grafico arancione, se usiamo Adam (Adaptive Moment Estimation), un ottimizzatore stocastico del primo ordine che aggiorna i pesi basandosi esclusivamente sulle stime adattive del gradiente e dei suoi momenti, la rete va in stallo: cade in un minimo locale e il parametro fallisce la convergenza, fermandosi lontano da 0.35.

La soluzione è passare all'algoritmo L-BFGS (Limited-memory Broyden-Fletcher-Goldfarb-Shanno), un metodo del secondo ordine appartenente alla famiglia dei Quasi-Newton. L-BFGS sfrutta l'approssimazione della matrice Hessiana (le derivate seconde) per decodificare la curvatura dello spazio e convergere al vero minimo globale. Trattandosi di un metodo 'Limited-Memory', esso stima l'inversa dell'Hessiana osservando solo la storia degli ultimi gradienti processati, evitando di allocare una matrice densa $O(N^2)$. Grazie a L-BFGS, aiutato da una funzione interna chiamata `closure()` per valutare step multipli di line-search, la curva verde crolla e la rete scopre perfettamente $D=0.3500$.

Ci tengo infine a precisare una distinzione fondamentale tra l'esecuzione in locale e sul cluster:

- **L'ambiente locale** è servito per il debug rapido e il testing iniziale del grafo differenziale con poche epoche (es. 100 epoche), utile per verificare la corretta compilazione del codice ma insufficiente a raggiungere la convergenza perfetta.

- **Il Cluster HPC dell'università**, invece, è stato impiegato per i test di scalabilità intensiva: i job

sono stati sottomessi tramite lo scheduler SLURM all'interno di container Apptainer con accelerazione GPU, macinando ben 50.000 epoche nominali. Questo ha permesso di testare la vera potenza di L-BFGS, che ha raggiunto la precisione assoluta sul Cluster in 131 secondi, validando definitivamente il modello in un ambiente di supercomputing."

Slide 11: I Risultati Finali

"Siamo giunti al confronto finale tra i due percorsi, analizzando in dettaglio i dati empirici raccolti dai grafici. Sotto il profilo dell'accuratezza (nel grafico a sinistra), registriamo un vero e proprio pareggio tecnico: sia il metodo numerico classico (Levenberg-Marquardt), sia le varianti Deep Learning con ottimizzatore L-BFGS (sia in ambiente locale che sul Cluster HPC) convergono con precisione chirurgica al valore target di D pari a 0.3500. Al contrario, l'ottimizzatore Adam (in rosso e viola) si ferma a stime viziate da errore, come 0.4501 o 0.3542, a causa della ben nota Gradient Pathology.

La vera dicotomia, visibile nel grafico di destra in scala logaritmica, emerge sui tempi computazionali. Il solutore numerico basato su matrici sparse CSR annienta l'approccio neurale, impiegando appena 0.05 secondi. Le PINN richiedono tempi enormemente superiori: da 1.07 e 5.93 secondi per i test locali leggeri, fino a 263.67 secondi per Adam e 131.22 secondi per L-BFGS sul Cluster GPU.

Tuttavia, un'analisi critica ci impone di valutare il costo computazionale sotto una duplice prospettiva: le PINN pagano un dazio computazionale estremo in fase di addestramento (Training Cost), ma una volta addestrate offrono un'inferenza istantanea di complessità $O(1)$ per qualsiasi predizione futura. L'approccio numerico, pur essendo fulmineo al momento della calibrazione, richiede di ricalcolare interamente i passi temporali e le inversioni di matrici ogni volta che ci si sposta nel tempo o si cambia scenario."

Slide 12: Conclusioni e Trade-Off

"Concludendo, questo progetto ha validato empiricamente due metodologie apparentemente antitetiche. Non esiste un 'vincitore assoluto', ma si aprono scenari applicativi ben definiti.

I Metodi Numerici Classici, esplorati nel Percorso A, si confermano superiori quando abbiamo a che fare con campionamenti uniformi e geometrie ben definite. L'algebra lineare iterativa offre convergenze in frazioni di secondo. Tuttavia, presentano due grandi limiti. Il primo è la cosiddetta 'maledizione della dimensionalità': scalano malissimo in alte dimensioni, perché passare da un

modello 2D a uno 3D farebbe esplodere la matrice, saturando la RAM. Il secondo limite è l'asimmetria strutturale tra problema diretto e problema inverso: per scovare il parametro D abbiamo dovuto stravolgere l'architettura di partenza, incapsulando il solutore iterativo dentro un ottimizzatore non lineare come Levenberg-Marquardt.

Le PINN, affrontate nel Percorso B, risolvono brillantemente entrambi questi ostacoli, offrendo una superiorità schiacciante quando i dati sono scarsi o frammentati. Grazie alla loro architettura mesh-free, sono immuni alla maledizione della dimensionalità: aggiungere un asse spaziale significa semplicemente fornire un input aggiuntivo alla rete neurale, senza dover allocare matrici gigantesche. Inoltre, l'architettura resta nativamente simmetrica: per risolvere il problema inverso non serve stravolgere il codice, è sufficiente impostare il coefficiente D come parametro ottimizzabile e la rete impara a calcolarlo simultaneamente alla dinamica diretta.

In sintesi, la Parameter Discovery spaziale può oggi beneficiare di due approcci profondamente diversi che, come dimostrato, sono in grado di validare indipendentemente il medesimo risultato termodinamico e fisico.

Grazie per l'attenzione."

Nota Finale / Possibile Domanda della Commissione: I Dati Reali vs. Soluzione Fabbricata

(Questa sezione può essere integrata alla fine delle conclusioni come "autocritica" o usata come risposta perfetta se il professore chiede perché si è usato $D=0.35$ invece dei dati grezzi del COVID-19).

"Vorrei fare un'ultima precisazione accademica, doverosa per onestà intellettuale, riguardo al dataset utilizzato per il benchmark finale.

Nel progetto completo, l'infrastruttura scarica e pulisce correttamente i dati reali del COVID-19 dalla Protezione Civile. Tuttavia, per il confronto ingegneristico tra i Metodi Numerici e le PINN, ho scelto volutamente di applicare il 'Metodo delle Soluzioni Fabbricate' (Method of Manufactured Solutions), testando i solutori su una realtà artificiale con $D=0.3500$.

Questa scelta architetturale è dettata da due precise motivazioni.

In primo luogo, un vero e proprio 'coefficiente di diffusione D ' spaziale del COVID-19 fisicamente non esiste. La diffusione reale è caotica, dipende da uso corretto mascherine, densità abitativa e lockdown. L'equazione del Laplaciano è un 'Toy Model', un'astrazione matematica che simula il virus come se fosse una diffusione di calore su una superficie piana.

In secondo luogo, se avessi dato in pasto agli ottimizzatori il rumore dei dati grezzi reali,

l'equazione non sarebbe mai riuscita a descrivere le impennate asimmetriche dei focolai: la Loss non sarebbe mai scesa a zero. L'algoritmo avrebbe restituito un numero, ma io non avrei avuto alcun 'Ground Truth' per misurarne il reale errore.

Usando un ambiente sterile di cui conoscevo in anticipo il risultato esatto, ho potuto misurare l'errore al decimillesimo. Questo è stato l'unico modo per generare i grafici di convergenza e dimostrare in modo matematicamente inattaccabile la 'Gradient Pathology', provando al di là di ogni dubbio che l'ottimizzatore L-BFGS è in grado di superare il minimo locale in cui Adam cade inesorabilmente.

In sintesi, trattandosi di un esame di Analisi Numerica Computazionale e non di Virologia, ho ritenuto prioritario validare il rigore e la stabilità degli algoritmi."